

博士研究开题报告

Language-Guided Verifiable Algorithm Unrolling for Operator-Splitting Optimisers

面向组合优化专业求解器的语言引导可验证算法展开

拟研究方向：大模型辅助优化算法设计、算法展开、组合优化专业求解器

2026 年 4 月 29 日

摘要

组合优化问题广泛存在于物流调度、能源系统、网络设计、生产计划、广告分配和芯片设计等真实工业场景中。现代专业求解器（如 SCIP、HiGHS、OR-Tools、Gurobi、CPLEX 等）通常依赖分支定界/分支切割、线性或二次松弛、启发式、切平面、预处理与局部搜索等模块协同工作。近年来，大模型在自动算法设计、组合优化启发式生成、程序搜索和反思式进化中显示出潜力；与此同时，将 PDHG、ADMM、proximal gradient 等经典 operator-splitting 优化算法展开为可学习迭代网络，也成为 learning-to-optimize 和 differentiable optimisation 的重要路线。然而，现有研究仍存在明显断层：大模型生成的组合优化算法多偏启发式，缺少与专业求解器内部松弛求解、证书和稳定性的深度结合；而现有 algorithm unrolling 工作多依赖人工设计结构，尚未形成面向组合优化专业求解器的可验证自动设计框架。

本开题报告提出一个博士研究课题：**Language-Guided Verifiable Algorithm Unrolling for Operator-Splitting Optimisers**。核心思想是：不让大模型直接求解组合优化问题，而让大模型作为离线算法设计器，在受约束的算法领域特定语言（DSL）中生成可验证的展开策略，包括步长调度、罚参数更新、restart 规则、残差平衡、混合算子切换和松弛-修复接口；随后通过静态验证器、理论安全类、执行反馈和多层 benchmark 对候选策略进行筛选。研究目标是将该框架嵌入组合优化专业求解器中的 LP/QP 松弛、拉格朗日松弛、Benders/ADMM 分解子问题和 LNS 子问题，形成兼具可解释性、可验证性、跨规模泛化和工程实用性的 LLM-guided unrolled optimiser。

预期贡献包括：(1) 建立面向 operator-splitting optimiser 的语言引导可验证算法搜索空间；(2) 提出大模型生成、验证器过滤、执行反馈进化和小规模到大规模迁移的闭环方法；(3) 给出安全策略类下的收敛性、残差稳定性、松弛误差传递和求解

器集成理论; (4) 构建面向组合优化专业求解器的系统 benchmark 与消融协议; (5) 在 MIPLIB、Distributional MIPLIB、ML4CO benchmark、CO-Bench/HeuriGym 相关任务及若干真实 MILP/CO 问题上验证方法效果。

目录

1	研究背景与意义	5
1.1	组合优化专业求解器的现实需求	5
1.2	大模型辅助算法设计的兴起	5
1.3	算法展开与 operator-splitting optimiser	6
2	相关工作综述	7
2.1	LLM 自动算法设计与组合优化	7
2.2	Algorithm unrolling 与 learned optimisation	7
2.3	ML for CO 与专业求解器增强	8
3	问题陈述与研究目标	8
3.1	核心研究问题	8
3.2	研究对象	9
3.3	总体目标	9
4	拟提出的方法框架	10
4.1	总体框架	10
4.2	Operator-splitting optimiser 模板	11
4.2.1	PDHG/PDLP 模板	11
4.2.2	ADMM 模板	11
4.2.3	Proximal gradient / FISTA 模板	11
4.3	算法 DSL 设计	12
4.4	Verifier 设计	12
4.5	Evaluator 设计	13
5	面向组合优化专业求解器的集成方案	13
5.1	集成层次	13
5.1.1	Level 1: 外部 relaxation accelerator	14
5.1.2	Level 2: 求解器 callback / primal heuristic	14
5.1.3	Level 3: 分解式子问题求解	14
5.2	与分支定界的安全接口	15

目录	3
6 理论研究计划	15
6.1 安全类下的 PDHG 稳定性	15
6.2 ADMM 罚参数策略的收敛稳定性	15
6.3 松弛误差向 MILP 求解器的传播	16
6.4 展开策略的跨规模泛化分析	16
7 实验设计	16
7.1 实验原则	16
7.2 Benchmark 选择	17
7.3 Baseline 设计	17
7.4 评价指标	18
7.5 关键实验	19
7.5.1 实验一: LLM 是否优于黑盒搜索?	19
7.5.2 实验二: 可验证约束是否必要?	19
7.5.3 实验三: 求解器集成收益	20
7.5.4 实验四: 跨规模泛化	20
7.5.5 实验五: 策略可解释性与失败分析	20
8 预期创新点	20
8.1 从 LLM heuristic generation 到 verifiable unrolled optimiser design	20
8.2 面向专业求解器的安全接口	21
8.3 可验证 DSL 与理论安全类	21
8.4 跨算法族统一视角	21
8.5 组合优化 benchmark 与 LLM cost-aware 评估	21
9 可行性分析	21
9.1 技术可行性	21
9.2 研究风险与应对	22
10 研究计划与时间安排	22
10.1 第一阶段: 文献、DSL 与基础原型 (第 1 年)	22
10.2 第二阶段: LLM 搜索闭环与理论安全类 (第 2 年)	22
10.3 第三阶段: 专业求解器集成与组合优化 benchmark (第 3 年)	23
10.4 第四阶段: 统一理论、真实应用与博士论文 (第 4 年)	23
11 拟投稿方向	23
12 预期成果	24

目录	4
13 结论	24

1 研究背景与意义

1.1 组合优化专业求解器的现实需求

组合优化 (Combinatorial Optimisation, CO) 问题通常可以通过混合整数线性规划 (MILP)、混合整数二次规划 (MIQP)、约束规划 (CP)、网络流、集合覆盖、车辆路径、最大独立集、调度和装箱等形式表达。以 MILP 为例, 其标准形式可写为:

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & c^\top x \\ \text{s.t.} \quad & Ax \leq b, \\ & \ell \leq x \leq u, \\ & x_i \in \mathbb{Z}, \quad i \in \mathcal{I}. \end{aligned} \tag{1}$$

现代专业求解器通常不会单靠一种算法, 而是将多个模块组合为复杂系统:

- **松弛求解器**: 在每个节点求解 LP/QP 松弛, 提供 lower bound、dual 信息、reduced cost 和分支信息;
- **分支定界/分支切割**: 通过分支、cut separation、domain propagation 和剪枝控制搜索树;
- **启发式与修复**: 产生高质量可行解, 改善 primal bound;
- **大邻域搜索 (LNS)**: 固定部分变量并重解子问题;
- **分解与并行化**: 对网络流、调度、能源系统等结构化问题使用 ADMM、Benders、Lagrangian relaxation 等。

这类专业求解器的优势在于可靠、可解释、具有证书; 劣势在于大量策略依赖人工经验, 且不同问题族的最优参数和策略差异显著。因此, 一个自然问题是:

能否利用大模型的程序生成和算法设计能力, 自动生成可验证、可集成、可迁移的优化算法展开策略, 从而增强专业组合优化求解器?

1.2 大模型辅助算法设计的兴起

近年来, LLM 不再只被用于自然语言任务, 也开始被用于程序搜索、数学发现和组合优化启发式生成。FunSearch 通过将预训练大模型与系统化 evaluator 结合, 在函数空间中搜索代码形式的解法, 强调 evaluator 对幻觉和错误方案的约束作用。EoH 使用大模型和进化计算共同演化启发式思想与可执行代码, 面向组合优化启发式自动设计。ReEvo 进一步将反思式语言反馈与进化搜索结合, 提出 LLM-based hyper-heuristics。LLM-LNS 将 LLM 用于 MILP 的大邻域搜索策略设计, 说明 LLM 设计求解器组件具

有现实潜力。CO-Bench 和 HeuriGym 则开始系统评估大模型在组合优化算法搜索与启发式生成上的能力。

这些工作说明：**LLM 作为算法设计器**是一条正在形成的研究路线。然而，当前 LLM-for-CO 研究主要集中在启发式、元启发式或完整代码生成，常见不足包括：

1. 输出空间过于开放，难以保证稳定性和可验证性；
2. 与专业求解器内部 LP/QP 松弛、dual bound、certificate、node processing 的结合不足；
3. 大模型贡献容易退化为昂贵的黑盒超参数搜索；
4. 缺少跨求解器、跨问题族、跨规模的系统评估协议。

1.3 算法展开与 operator-splitting optimiser

Algorithm unrolling/unfolding 将经典迭代算法的有限步展开成可学习结构。对优化算法而言，常见形式为：

$$z_{k+1} = \mathcal{T}_{\theta_k}(z_k; P), \quad k = 0, \dots, K-1, \quad (2)$$

其中 P 是问题实例， z_k 是算法状态， $\mathcal{T}_{\theta_k}$ 是带参数的更新算子。与纯黑盒神经网络不同，algorithm unrolling 具有结构清晰、可解释、易结合理论和可控制迭代步数等优点。

在组合优化专业求解器中，最适合展开的不是 simplex 或 branch-and-bound 主循环本身，而是以下 operator-splitting 模块：

- **PDHG / PDLP**：适合大规模 LP 松弛，矩阵向量乘友好，可部署于 GPU；
- **ADMM / Douglas–Rachford splitting**：适合可分结构、分解式约束和 Lagrangian relaxation；
- **Proximal gradient / FISTA**：适合正则化松弛、二次罚函数和连续 surrogate；
- **Augmented Lagrangian / SQP 子问题**：适合约束优化与 MIQP/NLP 松弛；
- **PCG/linear solver module**：适合 QP、Newton、Gauss–Newton 或 KKT reduced system 的内层线性求解。

这些算法天然拥有层状结构、残差信息、步长/罚参数/restart 规则和可验证的安全条件。因此，相比直接让 LLM 生成任意启发式，**在 operator-splitting optimiser 的受控搜索空间内使用 LLM 自动设计展开策略**更有希望形成兼具新意和可靠性的 AI 会议论文方向。

2 相关工作综述

2.1 LLM 自动算法设计与组合优化

FunSearch. FunSearch 将 LLM 与自动 evaluator 配对，在代码函数空间中进行演化搜索，强调“LLM 提出候选，evaluator 负责验证”的闭环范式。该思想对本课题具有基础性启发：LLM 不应直接被当作 solver，而应作为 candidate generator。

EoH 与 ReEvo. EoH 将启发式的自然语言思想与可执行代码同时进化；ReEvo 通过反思式语言反馈进一步提升 heuristic generation 的效率和质量。二者共同说明，LLM 可以利用已有算法知识生成有价值的组合优化启发式。然而，这类方法通常输出开放式 heuristic，缺少与 solver certificate 和 operator-splitting 更新条件的紧密绑定。

LLM-LNS. LLM-LNS 将 LLM 用于 MILP 中的大邻域搜索策略设计，与 Gurobi、SCIP、手工 LNS 和学习型 LNS 进行比较。该工作最接近本课题的“专业求解器组件设计”动机，但其核心仍偏 neighborhood selection heuristic，而不是面向 LP/QP 松弛或 operator-splitting unrolled optimiser 的可验证更新设计。

CO-Bench 与 HeuriGym. CO-Bench 和 HeuriGym 说明研究社区正在重视 LLM 在组合优化算法搜索中的系统评估。这对本课题的 benchmark 设计具有直接参考价值：必须设置明确目标函数、可执行反馈、质量指标、失败率和样本效率指标，避免只做 narrative demo。

2.2 Algorithm unrolling 与 learned optimisation

PDHG-Net. PDHG-Unrolled Learning-to-Optimize Method for Large-Scale LP 将 PDHG 展开为 PDHG-Net，并采用“两阶段”方法：先由网络生成近似解，再由 PDHG 精修。其重要启发是：AI 会议认可“经典一阶优化器 + 展开结构 + 可靠后处理”的混合求解模式。

DeepDistributedQP. DeepDistributedQP 将 OSQP/consensus 风格的 QP 求解器展开成深度架构，并通过 PAC-Bayes 理论给出泛化边界。该工作表明：operator-splitting optimiser 的 learned unfolding 可以与大规模 QP、跨规模泛化和理论保证结合。

Deep FlexQP. Deep FlexQP 通过 deep unfolding 加速 nonlinear programming 中的 QP 子问题，并强调可行性、松弛与求解器集成。该方向说明，unrolled solver 若想走向真实专业求解器，必须关注 infeasibility、constraint violation 和 solver-compatible certificates，而不仅是 objective loss。

2.3 ML for CO 与专业求解器增强

ML4CO 研究包括 branching policy learning、cut selection、node selection、primal heuristic、LNS、neural diving、neural rounding、solver configuration 和 instance embedding 等。已有方法多使用 GNN 表示变量-约束二部图，并学习求解器局部决策。与之相比，本课题更关注：

- 不直接替代 solver 的关键决策，而是设计可验证的松弛求解和分解优化模块；
- 不只学习数值参数，而是通过 LLM 生成结构化更新规则；
- 不追求端到端黑盒，而是保留 operator-splitting 的理论骨架；
- 强调与专业 solver 的安全接口：bound、feasibility、certificate、repair。

3 问题陈述与研究目标

3.1 核心研究问题

本课题拟研究如下核心问题：

如何利用大语言模型，在可验证的受控算法空间中自动设计 operator-splitting optimiser 的展开策略，并将其嵌入组合优化专业求解器，以提升松弛求解、启发式修复、分解子问题和 LNS 子问题的效率与泛化能力？

该问题可分解为四个研究问题：

RQ1: 搜索空间 如何定义适合 LLM 生成、可执行、可验证、可理论分析的 unrolled optimiser DSL？

RQ2: 安全验证 如何保证 LLM 生成的策略不破坏基本优化性质，如有界步长、残差稳定、proximal non-expansiveness、feasibility repair 和 bound validity？

RQ3: 求解器集成 如何将生成策略嵌入 MILP/CO solver 的 LP/QP 松弛、Lagrangian/ADMM 分解、LNS 子问题和 rounding/repair 模块？

RQ4: 实验评估 如何设计 benchmark 和 baseline，证明方法不是 prompt engineering、不是黑盒超参数搜索、不是只在 toy problem 上有效？

3.2 研究对象

本文将组合优化专业求解器中的候选模块抽象为：

$$\mathcal{S}(P; \pi) = \text{SolverCore}\left(P, \text{RelaxSolver}_\pi, \text{Heuristics}_\pi, \text{Repair}, \text{Certify}\right), \quad (3)$$

其中 P 为组合优化实例， π 是由 LLM 生成并通过 verifier 筛选的策略程序。 π 不直接输出解，而是输出以下组件之一或多个：

- PDHG/ADMM/FISTA 的步长、罚参数、momentum 和 restart 调度；
- 残差平衡规则和 adaptive scaling；
- operator switching 规则，例如从 PDHG 切换到 proximal refinement；
- LNS 中 destroy/repair 的 schedule，与松弛残差、reduced cost 或 incumbent improvement 绑定；
- rounding-aware penalty update 与 feasibility repair 接口；
- 可选的 PCG/linear solver restart、truncation 或 preconditioning policy。

3.3 总体目标

本课题的总体目标是提出一个可复用框架：

$$\boxed{\text{LLM} + \text{DSL} + \text{Verifier} + \text{Evaluator} + \text{Solver Integration}} \quad (4)$$

使得大模型生成的算法展开策略能够：

1. 在小规模实例上通过自动评估和反思进化快速筛选；
2. 在大规模和分布外实例上保持稳定；
3. 在专业求解器中提供可解释的加速或质量提升；
4. 在理论上满足某类安全条件；
5. 在评估上战胜手工规则、随机搜索、贝叶斯优化、进化搜索、常规 learned schedule 和 LLM heuristic generation baseline。

Algorithm 1 Language-Guided Verifiable Algorithm Unrolling

Require: Problem family $\mathcal{D}$, algorithm templates $\mathcal{A}$, DSL grammar $\mathcal{G}$, verifier $\mathcal{V}$, evaluator $\mathcal{E}$, LLM $\mathcal{M}$

- 1: Initialise candidate pool $\mathcal{P}_0$ with human-safe baseline policies
- 2: **for** $t = 0, 1, \dots, T - 1$ **do**
- 3: Select elite policies from $\mathcal{P}_t$ according to validation score
- 4: Prompt LLM $\mathcal{M}$ with problem description, DSL grammar, failure cases, and elite policies
- 5: Generate candidate policies $\{\pi_i\}_{i=1}^m \subset \mathcal{G}$
- 6: **for** each π_i **do**
- 7: Apply static verifier: type check, bound check, operator legality
- 8: Apply mathematical verifier: safe parameter ranges, monotonicity proxy, proximal validity
- 9: Apply smoke-test evaluator on small instances
- 10: **if** $\mathcal{V}(\pi_i) = \text{pass}$ **then**
- 11: Evaluate π_i on training instances using $\mathcal{E}$
- 12: Add π_i to candidate pool
- 13: **end if**
- 14: **end for**
- 15: Construct reflection messages from failures and successful cases
- 16: $\mathcal{P}_{t+1} \leftarrow \text{elites} + \text{diverse verified candidates}$
- 17: **end for**
- 18: Return best policies and optionally fine-tuned scalar constants

4 拟提出的方法框架

4.1 总体框架

整体流程如算法 1 所示。

框架关键点如下：

- **LLM 离线设计**：LLM 只在策略搜索阶段调用，部署求解时不调用 LLM；
- **受控 DSL**：LLM 只能生成有限、安全、可解释的策略程序；
- **多级 verifier**：先过滤不可运行和不安全策略，再进行 expensive benchmark；
- **执行反馈闭环**：候选策略通过真实优化指标排序，而不是通过语言判断；
- **求解器兼容**：最终策略必须能够嵌入专业 solver 的 relaxation、heuristic 或 LNS 模块。

4.2 Operator-splitting optimiser 模板

4.2.1 PDHG/PDLP 模板

考虑 saddle-point 形式:

$$\min_{x \in X} \max_{y \in Y} f(x) + \langle Kx, y \rangle - g^*(y). \quad (5)$$

标准 PDHG 更新为:

$$\begin{aligned} y^{k+1} &= \text{prox}_{\sigma_k g^*}(y^k + \sigma_k K \bar{x}^k), \\ x^{k+1} &= \text{prox}_{\tau_k f}(x^k - \tau_k K^\top y^{k+1}), \\ \bar{x}^{k+1} &= x^{k+1} + \theta_k(x^{k+1} - x^k). \end{aligned} \quad (6)$$

LLM 可设计:

$$(\tau_k, \sigma_k, \theta_k, \text{restart}_k, \text{rescale}_k) = \pi_{\text{PDHG}}(s_k), \quad (7)$$

其中 s_k 包括 primal residual、dual residual、duality gap proxy、objective change、iteration index、constraint violation 和 bound progress。

4.2.2 ADMM 模板

考虑可分问题:

$$\min_{x, z} f(x) + g(z) \quad \text{s.t.} \quad Bx + Cz = d. \quad (8)$$

ADMM 更新为:

$$\begin{aligned} x^{k+1} &= \arg \min_x f(x) + \frac{\rho_k}{2} \|Bx + Cz^k - d + u^k\|^2, \\ z^{k+1} &= \arg \min_z g(z) + \frac{\rho_k}{2} \|Bx^{k+1} + Cz - d + u^k\|^2, \\ u^{k+1} &= u^k + Bx^{k+1} + Cz^{k+1} - d. \end{aligned} \quad (9)$$

LLM 可生成 residual balancing 规则:

$$\rho_{k+1} = h_\pi(\rho_k, \|r_p^k\|, \|r_d^k\|, \Delta f_k, k), \quad (10)$$

并由 verifier 保证:

$$0 < \rho_{\min} \leq \rho_k \leq \rho_{\max} < \infty. \quad (11)$$

4.2.3 Proximal gradient / FISTA 模板

对连续松弛或正则化 surrogate:

$$\min_x F(x) = \phi(x) + \lambda R(x), \quad (12)$$

其中 ϕ 光滑, R 可 prox。基本更新为:

$$x^{k+1} = \text{prox}_{\eta_k \lambda R}(v^k - \eta_k \nabla \phi(v^k)). \quad (13)$$

LLM 可设计:

$$(\eta_k, \beta_k, \text{restart}_k, \text{rounding_trigger}_k) = \pi_{\text{prox}}(s_k). \quad (14)$$

4.3 算法 DSL 设计

DSL 的目标是限制 LLM 的输出空间, 使其既有表达能力, 又能被验证。一个策略程序可写为:

```
policy PDHG_Schedule:
  features:
    rp = norm(primal_residual)
    rd = norm(dual_residual)
    gap = gap_proxy
    prog = previous_gap - gap
  update:
    if rp > c1 * rd:
      tau = clip(tau * a1, tau_min, tau_max)
      sigma = clip(sigma / a1, sigma_min, sigma_max)
    elif rd > c2 * rp:
      tau = clip(tau / a2, tau_min, tau_max)
      sigma = clip(sigma * a2, sigma_min, sigma_max)
    if prog < eps and k > k_min:
      restart_momentum()
    enforce tau * sigma * K_norm_sq <= 1 - margin
```

DSL 包含:

- **状态特征:** 残差、gap proxy、bound progress、incumbent improvement、node depth、constraint violation;
- **安全算子:** clip、normalize、restart、damping、rescale、prox、projection;
- **模板变量:** $\tau, \sigma, \rho, \theta, \eta, \beta, K_{\text{inner}}$;
- **约束语句:** boundedness、step-size product、monotonic proxy、repair requirement;
- **求解器接口:** return relaxed solution, dual certificate, rounding hint, LNS neighbourhood score。

4.4 Verifier 设计

Verifier 分为四层。

第一层: 语法与类型验证。 检查 DSL 是否可解析, 变量是否定义, 算子是否在允许集合中, 输出是否符合求解器接口。

第二层：静态安全验证。 检查参数范围、步长积、罚参数上下界、restart 规则是否可能导致非法状态。例如 PDHG 安全条件：

$$\tau_k \sigma_k \|K\|^2 \leq 1 - \delta. \quad (15)$$

第三层：动态 smoke test。 在极小实例上运行若干步，检查是否出现 NaN、Inf、残差爆炸、不可恢复 infeasibility、时间超限或违反 solver API。

第四层：数学安全类验证。 将候选策略归入某个安全类 Π_{safe} 。例如：

$$\Pi_{\text{PDHG}} = \{\pi : \tau_k, \sigma_k > 0, \tau_k \sigma_k \|K\|^2 \leq 1 - \delta, \theta_k \in [0, 1], \forall k\}. \quad (16)$$

后续理论只需对 Π_{safe} 成立，而不必证明 LLM 本身可靠。

4.5 Evaluator 设计

候选策略评分函数应同时考虑质量、速度、稳定性和泛化。对单个实例 P ，可定义：

$$\text{Score}(\pi; P) = w_1 \cdot \text{Gap}(P, \pi) + w_2 \cdot \text{Viol}(P, \pi) + w_3 \cdot \text{Time}(P, \pi) + w_4 \cdot \text{Fail}(P, \pi) + w_5 \cdot \text{Cost}_{\text{LLM}}(\pi), \quad (17)$$

其中：

- Gap: objective gap、duality gap、integrality gap 或 primal integral;
- Viol: 约束违反、整数违反、可行性修复失败率;
- Time: wall-clock、迭代数、node time;
- Fail: 数值失败、超时、不可解析、solver crash;
- Cost_{LLM} : LLM 查询成本与候选生成次数。

对训练集 $\mathcal{D}_{\text{train}}$ ，总体评分为：

$$\text{Score}(\pi) = \mathbb{E}_{P \sim \mathcal{D}_{\text{train}}} [\text{Score}(\pi; P)] + \lambda \text{Complexity}(\pi), \quad (18)$$

其中 $\text{Complexity}(\pi)$ 惩罚过长、过深、不可解释的策略程序。

5 面向组合优化专业求解器的集成方案

5.1 集成层次

本课题不试图重写完整商业求解器，而采用由浅入深的三层集成路线。

5.1.1 Level 1: 外部 relaxation accelerator

将 MILP/CO 问题的 LP/QP 松弛导出为标准格式, 用 LLM-generated PDHG/ADMM/FISTA 策略加速松弛求解, 输出:

- relaxation objective bound;
- approximate primal/dual solution;
- reduced-cost-like score;
- rounding 或 branching hint。

该层最容易实现, 适合早期验证。

5.1.2 Level 2: 求解器 callback / primal heuristic

将生成策略嵌入 SCIP/OR-Tools/HiGHS 等开源求解器的 callback:

- 使用松弛解指导 rounding;
- 使用 residual/gap 信息触发 local repair;
- 使用 LNS schedule 选择需要释放的变量集合;
- 将候选解交给专业 solver repair, 保证最终可行性。

5.1.3 Level 3: 分解式子问题求解

对网络流、能源调度、设施选址、多商品流、车辆路径等结构化问题, 使用 ADMM/Benders/Lagrangian relaxation 构建分解求解器。LLM 设计:

- penalty update;
- block update order;
- restart/damping;
- subproblem time allocation;
- primal recovery 和 repair schedule。

5.2 与分支定界的安全接口

在专业 MILP solver 中，任何学习模块都不能破坏 soundness。因而本课题采用如下原则：

1. **不替代 pruning certificate**: 学习模块给出的 lower bound 若不满足严格认证，则只能作为启发式信号，不能用于剪枝；
2. **可行解必须由 solver 验证**: rounding 或 LNS 产生的解必须经过原始约束检查；
3. **松弛误差有记录**: approximate relaxation bound 必须携带 residual/gap certificate；
4. **失败可回退**: 任何 LLM-generated policy 失败时立即回退到 vanilla PDHG/ADMM/solver default。

6 理论研究计划

本课题的理论目标不是证明“大模型会生成好算法”，而是证明：**只要大模型生成的策略属于某个可验证安全类，则展开优化器保留基本性质；其近似误差在专业求解器中以可控方式传播。**

6.1 安全类下的 PDHG 稳定性

假设 1 (凸松弛与有界算子). 考虑凸-凹 *saddle-point* 问题， f 和 g^* 为闭真凸函数， K 为有界线性算子。

定义 1 (PDHG 安全策略类). 称策略 π 属于 $\Pi_{\text{PDHG}}(\delta)$ ，若其生成的参数满足：

$$\tau_k > 0, \quad \sigma_k > 0, \quad \theta_k \in [0, 1], \quad \tau_k \sigma_k \|K\|^2 \leq 1 - \delta, \quad \forall k. \quad (19)$$

定理 1 (安全类下的 ergodic gap 收敛目标). 若 $\pi \in \Pi_{\text{PDHG}}(\delta)$ ，则在标准 PDHG 假设下，展开迭代的 *ergodic primal-dual gap* 可由 $O(1/K)$ 控制；常数依赖于 δ 、初始距离和参数上下界。

备注 1. 该定理可作为第一阶段理论目标。实际论文中需处理 *adaptive* τ_k, σ_k 和 *restart* 对 *Lyapunov* 函数的影响。核心思想是 *verifier* 将 LLM 生成策略限制在不破坏 PDHG 基本收敛条件的区域内。

6.2 ADMM 罚参数策略的收敛稳定性

定义 2 (ADMM bounded penalty policy). 称 π 属于 $\Pi_{\text{ADMM}}(\rho_{\min}, \rho_{\max})$ ，若：

$$0 < \rho_{\min} \leq \rho_k \leq \rho_{\max} < \infty, \quad \forall k, \quad (20)$$

并且每次 ρ 变化时对 *scaled dual variable* 进行一致重标定。

定理 2 (变罚参数 ADMM 的残差稳定性目标). 在闭真凸、鞍点存在和子问题精确求解的条件下, 若 $\pi \in \Pi_{\text{ADMM}}$ 且参数变化总变差受控, 则 *primal residual* 和 *dual residual* 收敛到零, *objective value* 收敛到最优值。

该方向将为 LLM 设计 residual balancing rule 提供理论安全边界。

6.3 松弛误差向 MILP 求解器的传播

设某节点 LP 松弛最优值为 v^* , approximate relaxation solver 输出 $\hat{v}$ 与 gap certificate ϵ , 满足:

$$|\hat{v} - v^*| \leq \epsilon. \quad (21)$$

命题 1 (近似松弛 bound 的安全使用). 若当前 *incumbent* 为 U , 则只有当

$$\hat{v} - \epsilon \geq U \quad (22)$$

时, 才可安全剪枝; 若无法给出严格 ϵ , 则 $\hat{v}$ 只能用于 *node scoring*、*branching hint* 或 *heuristic*, 而不能用于 *correctness-critical pruning*。

该命题体现专业求解器集成的基本原则: 学习模块可以增强效率, 但不能破坏求解器正确性。

6.4 展开策略的跨规模泛化分析

对于策略程序 π , 其复杂度可以用 DSL 长度、条件分支数、连续参数数目和算子组合深度度量。可考虑 PAC-Bayes 或 algorithmic stability 风格的泛化界:

$$\mathbb{E}_{P \sim \mathcal{D}} [\ell(\pi; P)] \leq \frac{1}{N} \sum_{i=1}^N \ell(\pi; P_i) + \mathcal{O} \left(\sqrt{\frac{\text{Complexity}(\pi) + \log(1/\delta)}{N}} \right). \quad (23)$$

该方向可解释为什么受控 DSL 比开放式代码生成更容易泛化。

7 实验设计

7.1 实验原则

为避免“纯讲故事”, 实验必须遵循以下原则:

1. **固定查询预算**: LLM、进化搜索、随机搜索、BO 使用相同候选评估预算;
2. **训练/验证/测试分离**: 小规模实例用于搜索, 大规模和分布外实例用于最终评估;
3. **报告失败率**: 包括不可运行、验证失败、数值爆炸、超时和可行性失败;

- 4. **报告求解器指标**: 不仅报告 loss, 还报告 gap、time、node count、primal integral、feasibility 和 bound;
- 5. **消融 LLM 的贡献**: 比较 LLM、随机生成、进化无 LLM、BO、手工规则和 learned controller。

7.2 Benchmark 选择

阶段	数据/问题	目的
Stage 1	合成 LP/QP、LASSO、network flow、assignment relaxation	快速验证 DSL、verifier、PDHG/ADMM/FISTA 策略是否有效
Stage 2	MIPLIB 2017、Distributional MIPLIB、ML4CO competition-style MILP distributions	测试与专业 MILP solver 相关的 relaxation、rounding、LNS、node-level 指标
Stage 3	set cover、facility location、knapsack、MIS、TSP/VRP MILP formulation	测试结构化 CO 问题上的泛化和 solver integration
Stage 4	CO-Bench、HeuriGym 相关任务	与 LLM heuristic generation benchmark 对齐, 展示本方法不是普通 LLM heuristic
Stage 5	真实应用案例: 物流、能源调度、广告分配或网络设计	检验工程价值与鲁棒性

7.3 Baseline 设计

Baseline 类型	具体方法
专业求解器	Gurobi、CPLEX、SCIP、HiGHS、OR-Tools PDLP/PDHG、OSQP
原始算法	vanilla PDHG、vanilla ADMM、FISTA、Douglas–Rachford、solver default LNS
手工自适应规则	residual balancing ADMM、adaptive restart FISTA、hand-crafted PDHG restart/scaling

黑盒搜索	random search、grid search、Bayesian optimisation、CMA-ES、evolutionary search without LLM
学习型 unrolling	learned scalar schedule、MLP/LSTM controller、hypernetwork-generated parameters、GNN policy
LLM 算法设计	FunSearch-style program search、EoH-style heuristic evolution、ReEvo-style reflective evolution、LLM-LNS-style neighbourhood design
消融版本	no verifier、no reflection、no evolution、no DSL constraints、LLM-only、small-scale only、without solver repair

7.4 评价指标

松弛求解指标

- primal residual、dual residual、duality gap；
- time-to- ϵ gap；
- fixed-budget final gap；
- GPU/CPU wall-clock；
- memory footprint。

MILP/CO 求解器指标

- primal bound、dual bound、integrality gap；
- primal integral；
- number of explored nodes；
- time to first feasible solution；
- time to best incumbent；
- final optimality gap under time limit；
- feasibility violation 和 repair success rate。

LLM algorithm design 指标

- valid program rate;
- verified safe program rate;
- query budget;
- candidate evaluation budget;
- best score vs. number of generated candidates;
- transfer performance from small to large instances;
- diversity and interpretability of discovered policies。

7.5 关键实验

7.5.1 实验一：LLM 是否优于黑盒搜索？

在固定候选数量 B 下比较：

$$\text{LLM+Verifier+Evolution} \quad \text{vs.} \quad \text{Random DSL Search} \quad \text{vs.} \quad \text{BO/CMA-ES} \quad \text{vs.} \quad \text{Human Rule.} \quad (24)$$

核心指标为 best verified score、valid rate 和大规模迁移表现。

7.5.2 实验二：可验证约束是否必要？

比较：

- full method;
- no static verifier;
- no mathematical safety constraints;
- open-ended code generation;
- LLM natural language prompt only。

预期验证器能显著降低 crash rate 和 OOD failure rate。

7.5.3 实验三：求解器集成收益

在 MIPLIB/Distributional MIPLIB 子集上测试：

- default solver;
- solver + vanilla relaxation accelerator;
- solver + hand-crafted unrolled strategy;
- solver + LLM-generated verified strategy。

指标包括 time-to-incumbent、gap、node count、primal integral。

7.5.4 实验四：跨规模泛化

在小规模实例上搜索策略，在大规模实例上测试：

$$n_{\text{train}} \ll n_{\text{test}}, \quad m_{\text{train}} \ll m_{\text{test}}. \quad (25)$$

分析策略是否依赖绝对维度，或只依赖归一化残差和比例特征。

7.5.5 实验五：策略可解释性与失败分析

对最优策略进行人工解释：

- 它是否类似已知 residual balancing 或 restart 规则？
- 它在哪些实例族上失败？
- 失败是否由 ill-conditioning、infeasibility、弱松弛、整数结构或 solver callback overhead 引起？

8 预期创新点

8.1 从 LLM heuristic generation 到 verifiable unrolled optimiser design

现有 LLM-for-CO 多生成启发式或完整代码。本课题强调生成的是 operator-splitting optimiser 的结构化展开策略，并由 verifier 和理论安全类约束。这使得方法更适合专业求解器，而不是只做开放式 heuristic search。

8.2 面向专业求解器的安全接口

本课题明确区分：

- correctness-critical 信息：必须由严格 solver 或 certificate 保证；
- heuristic information：可由学习模块提供，用于 node scoring、branching hint、rounding、LNS；
- approximate relaxation 信息：必须携带 residual/gap 证书。

这使得 LLM-generated optimiser 能安全嵌入专业 solver。

8.3 可验证 DSL 与理论安全类

通过 DSL 控制表达空间，并将候选策略映射到 Π_{safe} ，避免开放式代码生成的不可控性。理论上证明的是安全类性质，而不是 LLM 本身。

8.4 跨算法族统一视角

将 PDHG、ADMM、FISTA、Douglas–Rachford、PCG 等算法统一为：

$$z_{k+1} = \mathcal{T}_{a_k, \theta_k}(z_k; P), \quad (26)$$

其中 LLM 生成的是 a_k 和 θ_k 的策略程序。这比单一算法 unrolling 更具有博士研究容量。

8.5 组合优化 benchmark 与 LLM cost-aware 评估

本课题将专业求解器指标与 LLM algorithm design 指标结合，报告 valid rate、query cost、candidate budget、OOD failure 等，使得可行性论证更扎实。

9 可行性分析

9.1 技术可行性

- PDHG/ADMM/FISTA 等算法已有成熟理论和开源实现，适合作为第一阶段模板；
- SCIP、HiGHS、OR-Tools、Pyomo、JuMP 等生态支持求解器集成和 benchmark；
- LLM program search 可以先基于 API 调用和离线执行，不需要训练大模型；
- DSL 限制输出空间，降低 hallucination 和不可运行风险；
- benchmark 可从合成实例逐步扩展到 MIPLIB 和 Distributional MIPLIB。

9.2 研究风险与应对

风险	应对方案
LLM 生成策略不优于随机搜索	设置相同预算下的 random/BO/evolution baseline; 强调 LLM 是否提升 sample efficiency 和 valid rate
开放式代码不可控	使用 DSL、static verifier 和安全类理论; 禁止任意 Python 代码进入 solver core
求解器集成过难	先做 external relaxation accelerator, 再做 call-back/LNS, 最后考虑深度集成
理论只能覆盖凸松弛	明确理论对象为 LP/QP/convex surrogate; 对整数层只证明安全接口和误差传播
效果只在小规模有效	从一开始设计 train-small-test-large 和 OOD benchmark
LLM cost 过高	离线生成, 部署不调用 LLM; 报告 query budget 和 amortised cost
审稿人认为是 prompt engineering	公开 DSL、evaluator、query logs、候选池、失败案例和固定预算协议

10 研究计划与时间安排

10.1 第一阶段：文献、DSL 与基础原型（第 1 年）

- 系统整理 LLM-for-CO、algorithm unrolling、operator-splitting、ML4CO solver enhancement 文献;
- 实现 PDHG、ADMM、FISTA 三个基础模板;
- 设计第一版 DSL 和 static verifier;
- 在 LASSO、network flow、assignment relaxation 和合成 QP 上验证;
- 目标产出：一篇 workshop/CCF-B 论文或技术报告。

10.2 第二阶段：LLM 搜索闭环与理论安全类（第 2 年）

- 实现 LLM + reflection + evolutionary search;
- 完成 dynamic verifier 和 evaluator;

- 形成 PDHG/ADMM 安全策略类理论；
- 与 random search、BO、evolution、learned controller 系统对比；
- 目标产出：AAAI/IJCAI/ICLR 投稿。

10.3 第三阶段：专业求解器集成与组合优化 benchmark（第 3 年）

- 集成 SCIP/HiGHS/OR-Tools 或 Gurobi callback；
- 在 MIPLIB、Distributional MIPLIB、ML4CO-style distributions 上测试；
- 设计 LNS/rounding/repair 接口；
- 完成跨规模、跨问题族泛化实验；
- 目标产出：ICLR/ICML/NeurIPS 或高水平 OR/AI 交叉会议论文。

10.4 第四阶段：统一理论、真实应用与博士论文（第 4 年）

- 完善泛化理论、误差传播理论和 solver soundness 接口；
- 在真实工业问题上验证；
- 开源框架与 benchmark 协议；
- 汇总形成博士论文。

11 拟投稿方向

- **ICLR**: 最适合 language-guided unrolled optimiser、结构化学习求解器、跨规模泛化；
- **ICML/NeurIPS**: 适合理论更强的 safe policy class、泛化、differentiable optimisation 和算法设计；
- **AAAI/IJCAI**: 适合 solver-integrated LLM algorithm design 和组合优化应用；
- **CPAIOR/INFORMS/Mathematical Programming Computation**: 适合更偏专业求解器和运筹优化的版本；
- **NeurIPS/ICLR workshop**: 适合早期 benchmark、DSL 和系统原型。

12 预期成果

1. 一个可复用的 **Language-Guided Verifiable Unrolling** 框架；
2. 一个面向 operator-splitting optimiser 的 DSL 和 verifier；
3. 若干被 LLM 发现的可解释 PDHG/ADMM/FISTA/LNS 策略；
4. 安全策略类下的收敛性、稳定性和误差传播理论；
5. 面向组合优化专业求解器的 benchmark 协议；
6. 开源代码、query logs、candidate library 和实验脚本；
7. 2-4 篇高水平论文，包括至少一篇 AI 顶会或 CCF-A 会议目标论文。

13 结论

本课题的核心判断是：单纯让 LLM 直接求解组合优化问题，容易退化为黑盒启发式；单纯做某个经典优化算法的展开，又可能缺少会议新意。更有潜力的方向是将二者结合：让 LLM 在受控、可验证、可理论分析的算法空间中，离线设计专业求解器可使用的 operator-splitting unrolled optimiser 组件。

该方向具有三方面优势：

- **方法新意**：连接 LLM 自动算法设计与 algorithm unrolling；
- **理论抓手**：通过 safe policy class 分析收敛、残差稳定和误差传播；
- **工程价值**：面向组合优化专业求解器中的 LP/QP 松弛、LNS、分解子问题和 repair 模块。

因此，本课题既具有博士研究的深度和持续性，也具有面向 AI 会议和专业优化社区的双重投稿潜力。

参考文献

- [1] B. Romera-Paredes, M. Barekatin, A. Novikov, et al. Mathematical discoveries from program search with large language models. *Nature*, 2024.
- [2] F. Liu, X. Tong, M. Yuan, X. Lin, F. Luo, Z. Wang, Z. Lu, and Q. Zhang. Evolution of heuristics: Towards efficient automatic algorithm design using large language model. *Proceedings of ICML*, 2024.
- [3] H. Ye, J. Wang, Z. Cao, F. Berto, C. Hua, H. Kim, J. Park, and G. Song. ReEvo: Large language models as hyper-heuristics with reflective evolution. *Proceedings of NeurIPS*, 2024.
- [4] Large Language Model-driven Large Neighborhood Search for Large-Scale MILP Problems. *ICML*, 2025.
- [5] B. Li, L. Yang, Y. Chen, S. Wang, Q. Chen, H. Mao, Y. Ma, A. Wang, T. Ding, J. Tang, and R. Sun. PDHG-Unrolled Learning-to-Optimize Method for Large-Scale Linear Programming. *Proceedings of ICML*, 2024.
- [6] A. D. Saravanos, H. Kuperman, A. Oshin, A. T. Abdul, V. Pacelli, and E. A. Theodorou. Deep Distributed Optimization for Large-Scale Quadratic Programming. *ICLR*, 2025.
- [7] A. Oshin, R. V. Ghosh, A. D. Saravanos, and E. A. Theodorou. Deep FlexQP: Accelerated Nonlinear Programming via Deep Unfolding. *ICLR*, 2026.
- [8] W. Sun, S. Feng, S. Li, and Y. Yang. CO-Bench: Benchmarking Language Model Agents in Algorithm Search for Combinatorial Optimization. *Proceedings of AAAI*, 2026.
- [9] H. Chen, Y. Wang, Y. Cai, H. Hu, J. Li, S. Huang, C. Deng, R. Liang, S. Kong, H. Ren, S. Samaranayake, C. Gomes, and Z. Zhang. HeuriGym: An Agentic Benchmark for LLM-Crafted Heuristics in Combinatorial Optimization. *ICLR Benchmark Track*, 2026.
- [10] A. Gleixner, G. Hendel, G. Gamrath, et al. MIPLIB 2017: Data-driven compilation of the 6th mixed-integer programming library. *Mathematical Programming Computation*, 2021.
- [11] D. Applegate, M. Díaz, H. Lu, and M. Lubin. Practical Large-Scale Linear Programming using Primal-Dual Hybrid Gradient. *NeurIPS*, 2021.

- [12] A. Chambolle and T. Pock. A first-order primal-dual algorithm for convex problems with applications to imaging. *Journal of Mathematical Imaging and Vision*, 2011.
- [13] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein. Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers. *Foundations and Trends in Machine Learning*, 2011.
- [14] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd. OSQP: An operator splitting solver for quadratic programs. *Mathematical Programming Computation*, 2020.
- [15] M. Gasse, D. Chételat, N. Ferroni, L. Charlin, and A. Lodi. The Machine Learning for Combinatorial Optimization Competition. *Proceedings of Machine Learning Research*, 2022.